Artificial Intelligence for Digital Characters

Reinforcement Learning for Behavior Synthesis

Dr. Yan Zhang

Meshcapade

https://yz-cnsdqz.github.io
https://meshcapade.com/

The three categories:

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning (RL)

Supervised Learning

- (data, label) pairs are given for learning.
- Labels are normally from human annotations.
- The model takes data as input and predicts its label.

Example: Pose Estimation

- (image, keypoint annotation) pairs are given for learning.
- The model takes an image as input and predicts the keypoints.
- In case of neural networks, back-propagation can be directly applied.

https://cocodataset.org/#keypoints-2020

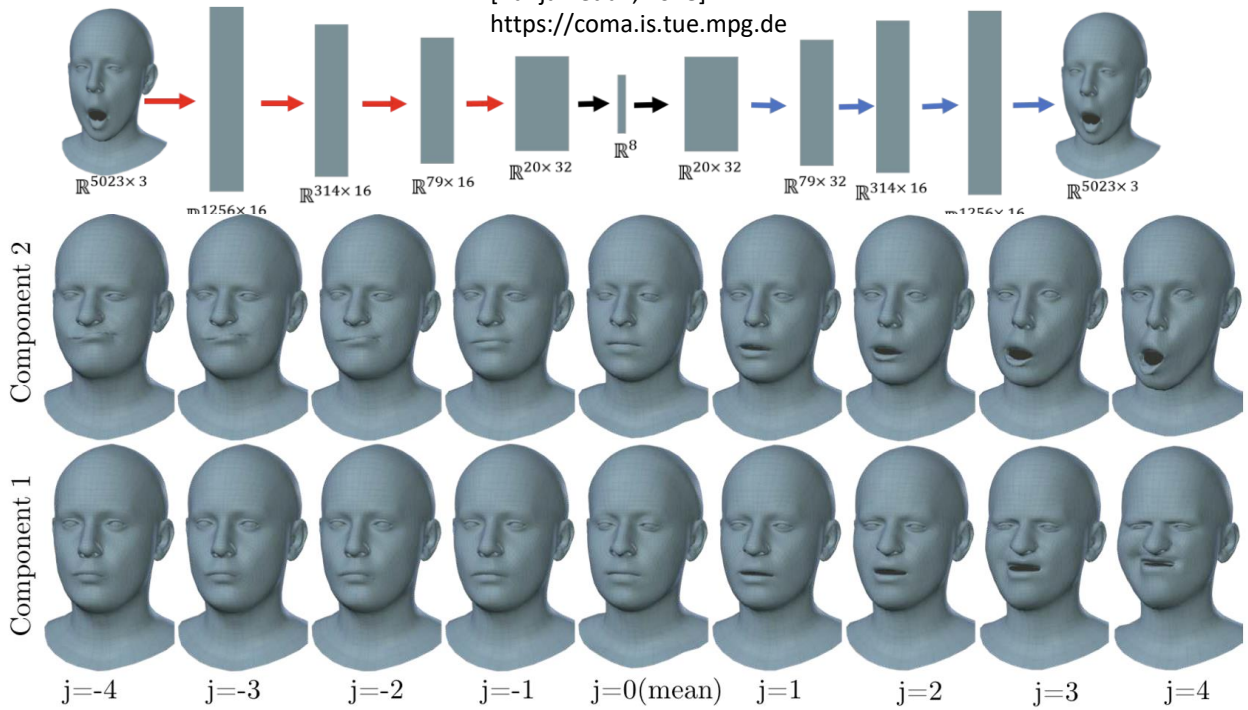
Unsupervised Learning

- Only data samples are given for learning.
- The model learns latent representations of the data.
- Self-supervised learning is a kind of unsupervised learning.

Example: Face Mesh Animation

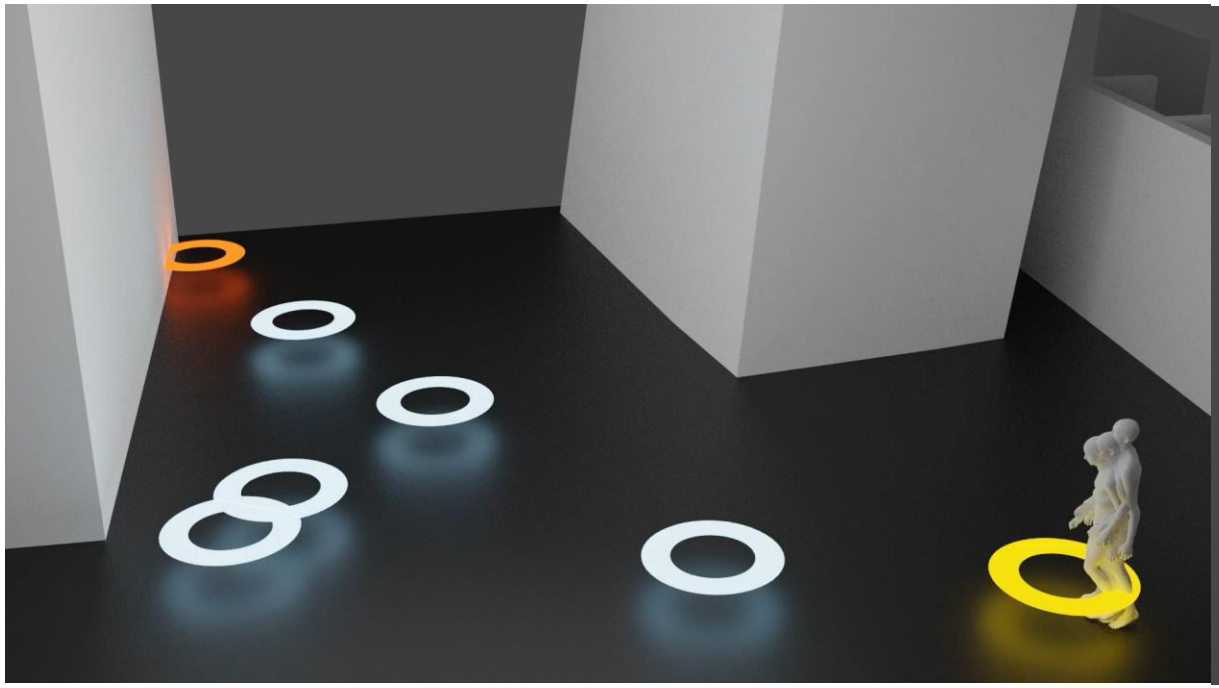
- Only face meshes are given for learning.
- The autoencoder model learns to reconstruct the mesh.
- Facial expression can be controlled by model latent variables.

[Ranjan et al., 2018]
https://coma.is.tue.mpg.de



Machine Learning

- What is reinforcement learning?
- How to model and synthesize human behaviors?



[Zhang and Tang, GAMMA, 2022]

[Zhao et al., DIMOS, 2023]

Key Concepts of RL

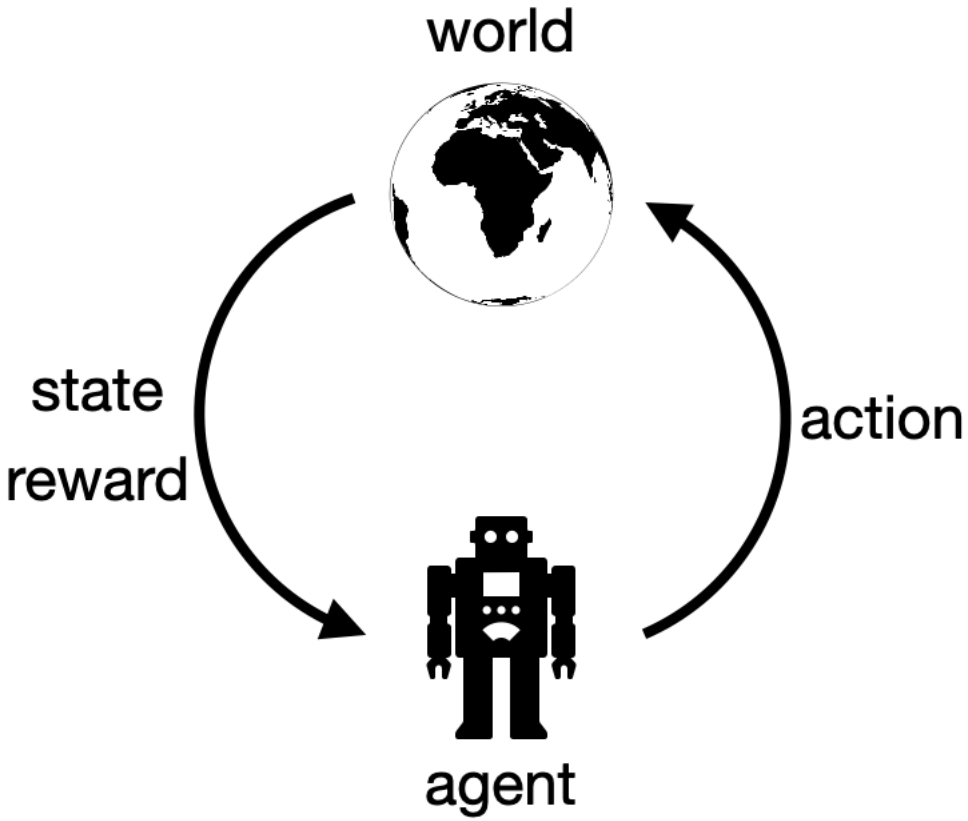
RL Learning Algorithms

Behavior Synthesis

Markov Decision Process
Trajectory
State & Action
Policy
Reward & Return
RL Objective
Value Functions

RL in a nutshell

- Learning is performed by agent-world interactions.
- Via interactions, data is collected.
- Based on the collected data, the agent learns how to perform better.



An example in DOTA 2

https://www.youtube.com/watch?v=SgWSdNh0C3s

state

action

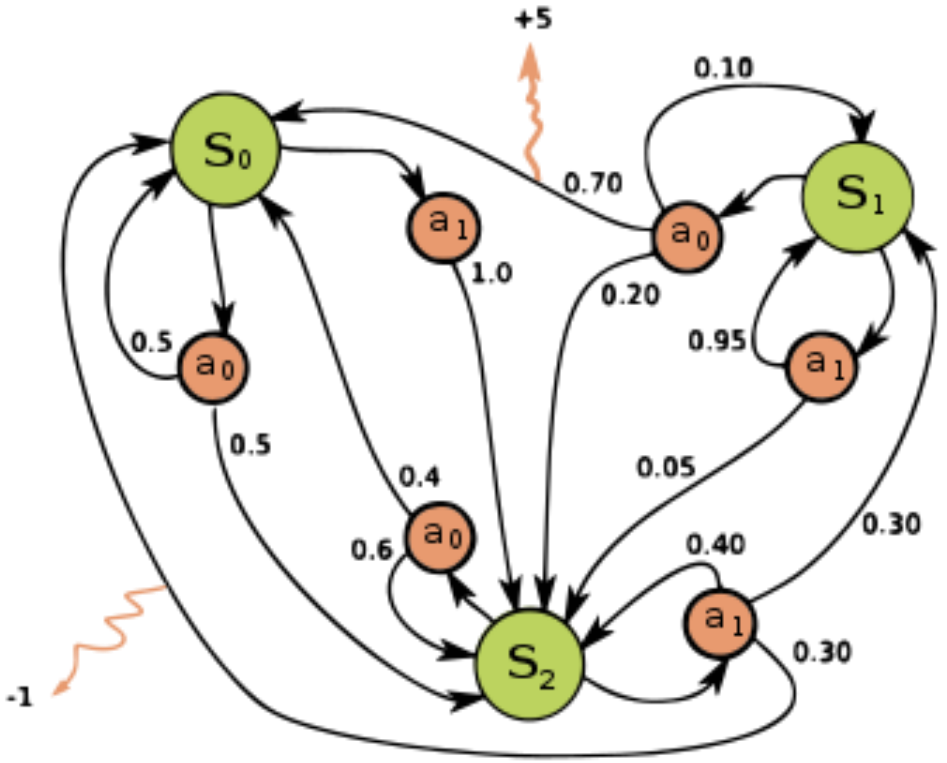
reward

Key Concepts in RL

- Markov Decision Process (MDP)
- RL trajectory
- State and action
- Policy
- Transition probability
- Reward and return
- The RL objective
- Value functions and Bellman equations
- …

Markov Decision Process (MDP)

- A tuple of $\langle S, A, R, P \rangle$, denoting the state space, action space, reward, and transition probability.
- If the action keeps constant and reward is always the same, MDP degenerates to a Markov chain.
- Markov property holds: The next state is only determined by the current state and action.



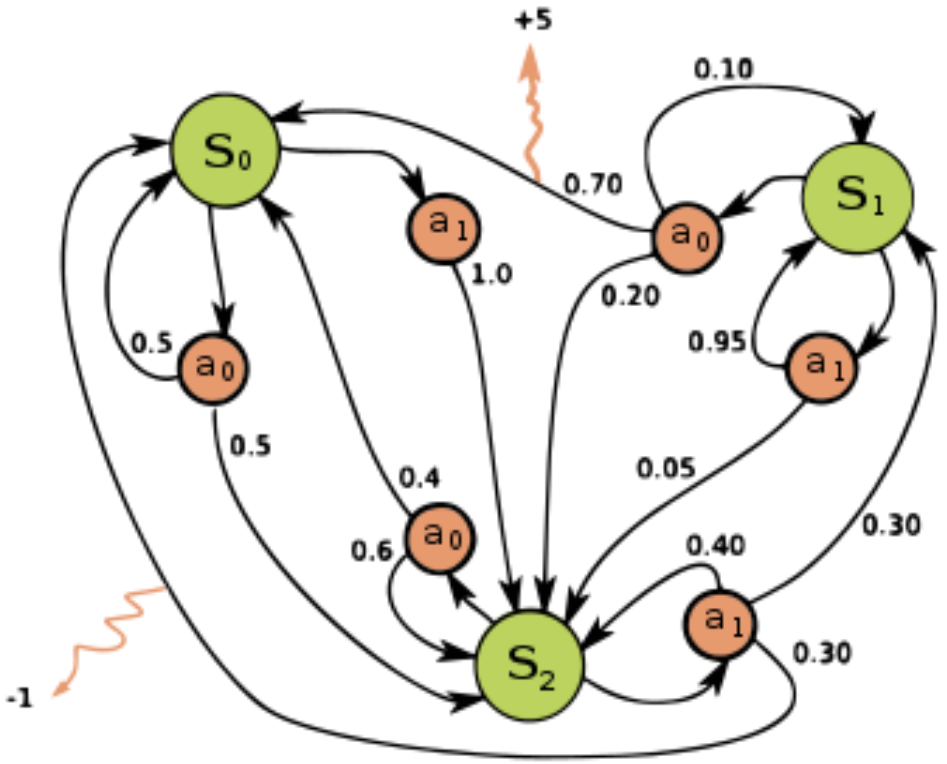
https://en.wikipedia.org/wiki/Markov_decision_process

RL trajectory

- RL trajectories are the data collected from agent-world interactions.

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_t, a_t, \ldots)$$

- RL trajectory is sometimes called episode or rollout.



https://en.wikipedia.org/wiki/Markov_decision_process

State & Action

State

- a complete description of the state of the world.
- The definition of the world is scenario-specific.
- E.g.: body poses, image pixels, distance to sofa, etc.

Action

- What the agent can do in the defined world.
- Discrete action spaces: positions in Go, pressed buttons in Super Mario.
- Continuous action spaces: robot arm joint torques, neural network latent variables.

State & Action

state

action

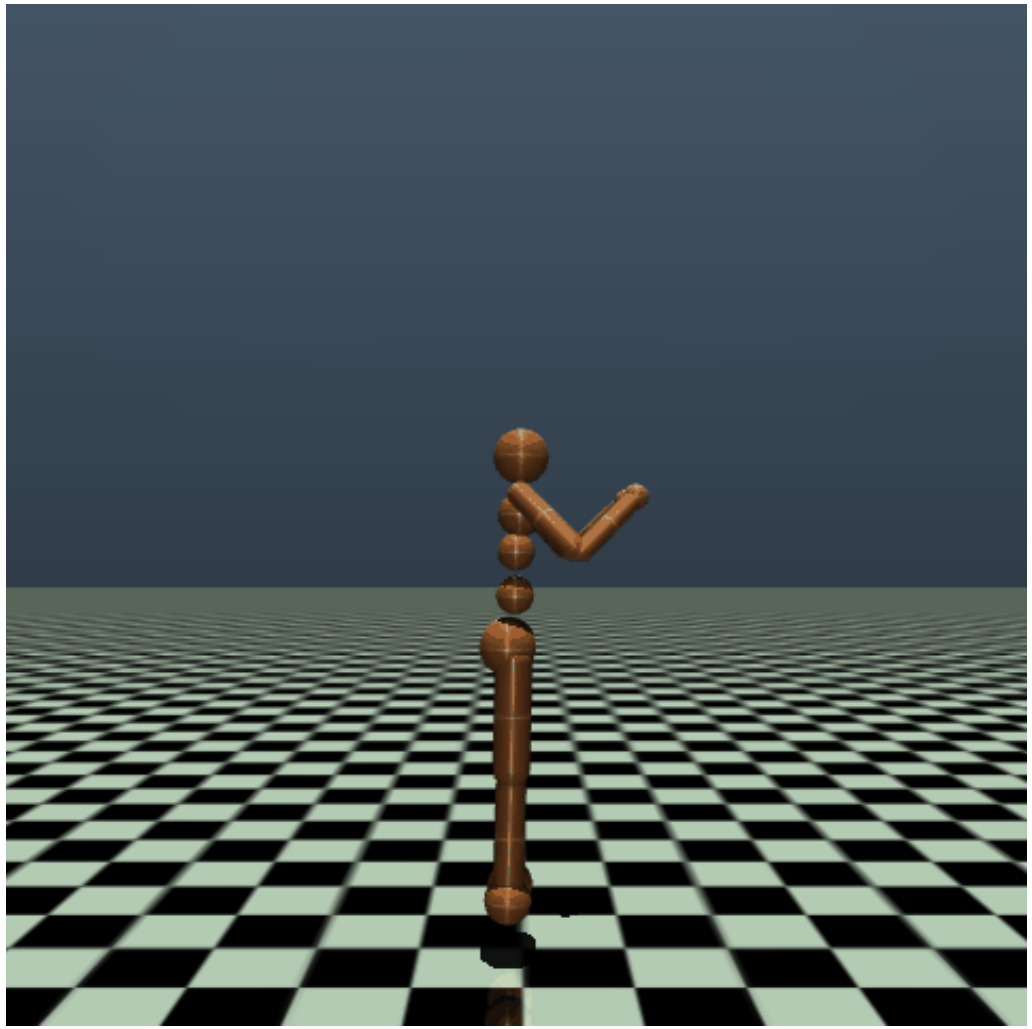
- According to the state, one needs to decide what to act.
- This is a decision process. This is the behavior of the player.

- Policy
  - The policy is like the agent brain, determining the agent behavior.
  - It takes a state as input and produces an action as output.
  - Depending on the uncertainty, a policy can be a deterministic function, or a probability. $a_t = \mu(s_t)$ or $a_t \sim \pi(\cdot|s_t)$
  - Depending on the action space continuity, the policy can be a regressor (for continuous space), or a classifier (for discrete space).

Transition probability

- The natural rule of the world, instead of the agent.

$$p(s_{t+1}|s_t, a_t)$$



https://gymnasium.farama.org/environments/mujoco/humanoid/

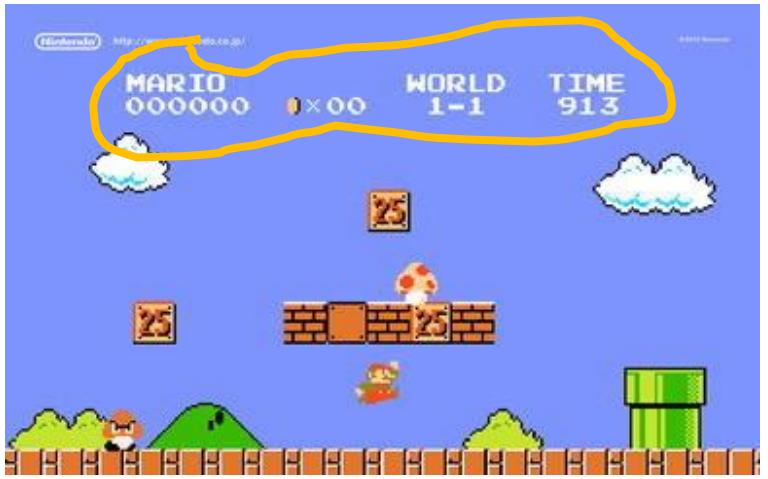
Reward

- The reward defines the goal and evaluates the current agent performance.

$$r_t = r(s_t, a_t)$$

- The reward is not necessary to be differentiable.
- Very essential in practice.

Return



- The return is defined on the entire trajectory.

$$R(\tau) = \sum_{t=0}^{T} \gamma^t r_t \quad \gamma \in (0, 1)$$

- The discount factor suggests rewards in the near future are preferred.

The RL objective: learn a policy to maximize the expected return.

- A RL trajectory conditioned on a policy

$$p(\tau|\pi) = p(s_0) \prod_t p(s_{t+1}|s_t, a_t)\pi(a_t|s_t)$$

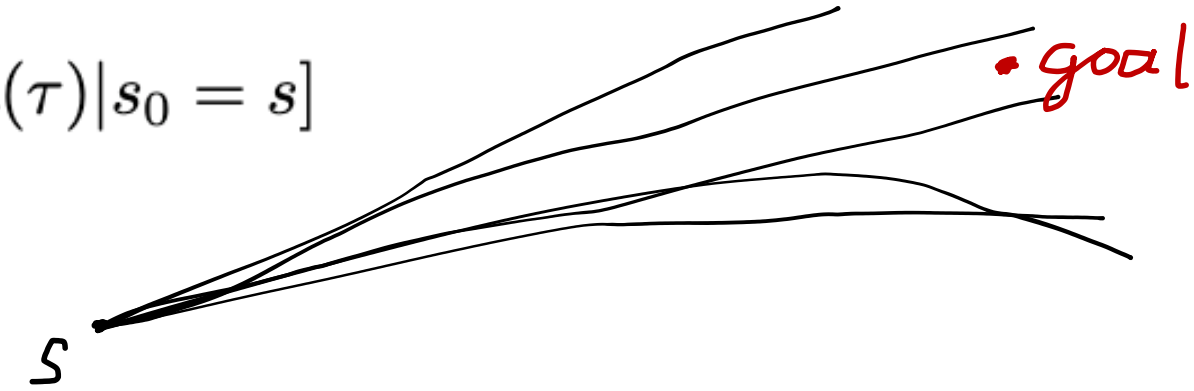
- The expected return

$$\mathbb{E}_{\tau\sim\pi}[R(\tau)] = \int_\tau p(\tau|\pi)R(\tau)d\tau$$

Value functions

- The value function estimates how well the agent will perform in the future.
- The first kind: State value function

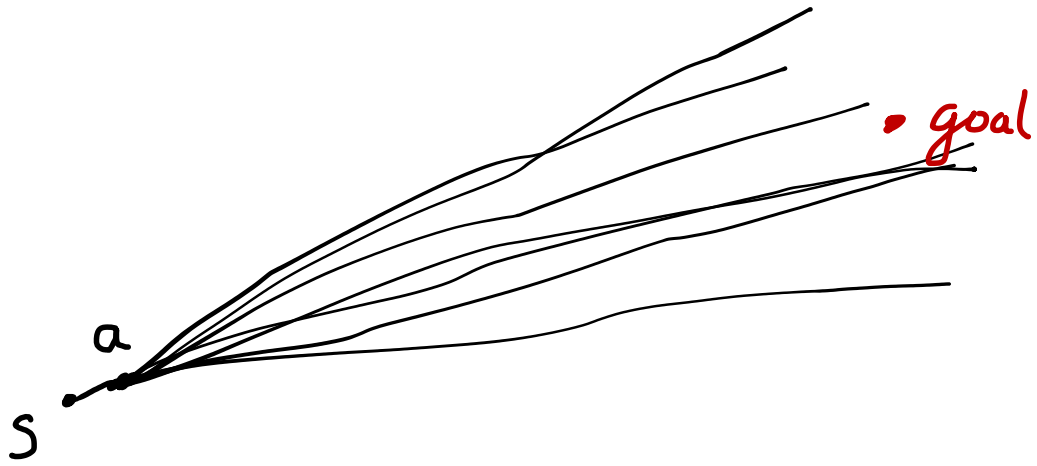
$$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s]$$



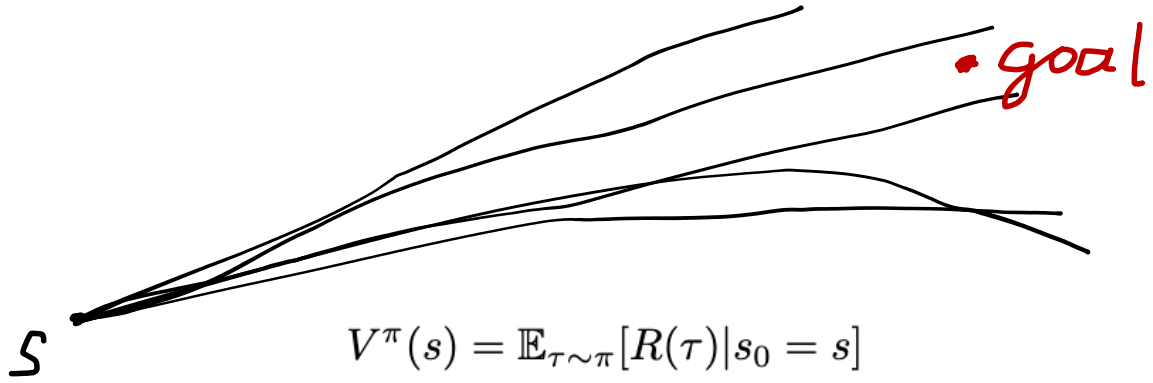
Value functions

- The second kind: state action value function, or Q-function.

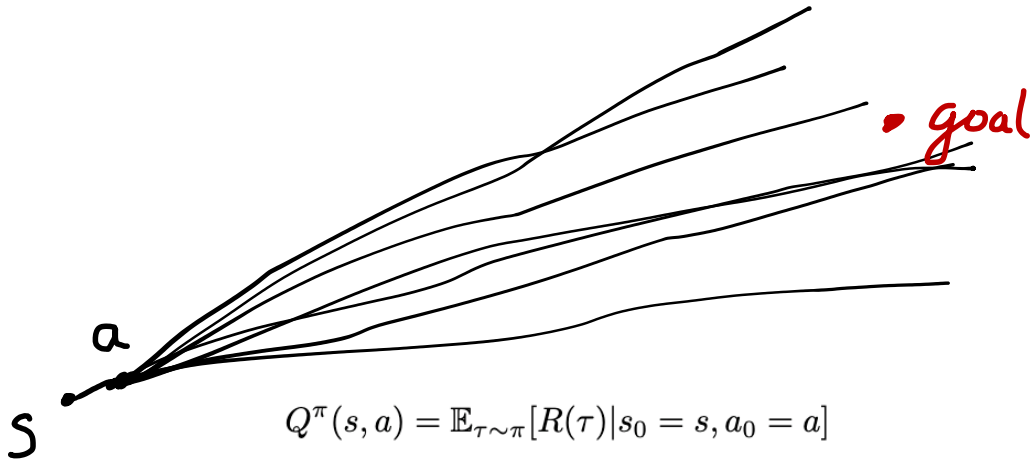
$$Q^{\pi}(s,a) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s, a_0 = a]$$



Value functions



$$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s]$$



$$Q^{\pi}(s,a) = \mathbb{E}_{\tau \sim \pi}[R(\tau)|s_0 = s, a_0 = a]$$

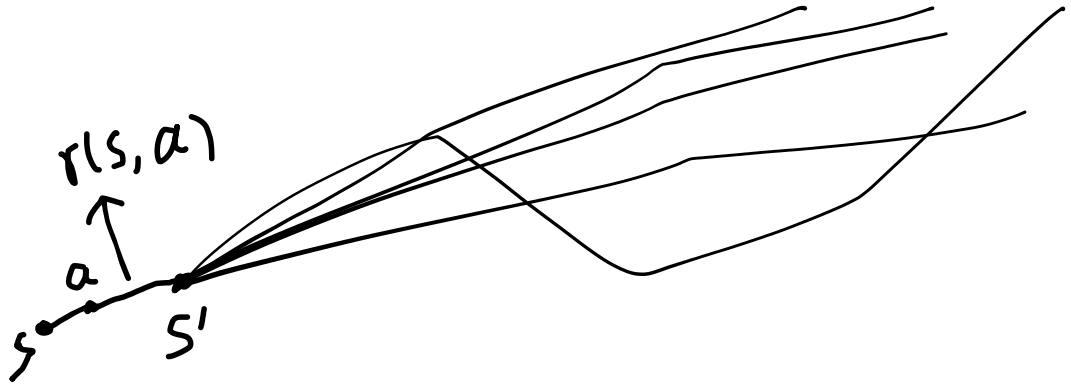
$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^{\pi}(s,a)]$$

Bellman equations

- The Bellman equations tell the **self-consistency between two consecutive time steps.**

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi, s'\sim p(\cdot|s,a)}[r(s,a) + \gamma V^{\pi}(s')]$$

$$Q^{\pi}(s,a) = r(s,a) + \gamma \mathbb{E}_{s'\sim p(\cdot|s,a), a'\sim\pi}[Q^{\pi}(s',a')]$$



Optimal value functions

- If the policy is optimal, the expected return is maximized at each individual step.
- Therefore, at each time step, both V() and Q() are maximized.

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^{\pi}(s,a)] \quad \Rightarrow \quad V^{*}(s) = \max_{a} Q^{*}(s,a)$$

Bellman equations in optimum

$$Q^{\pi}(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,a), a' \sim \pi}[Q^{\pi}(s',a')]$$

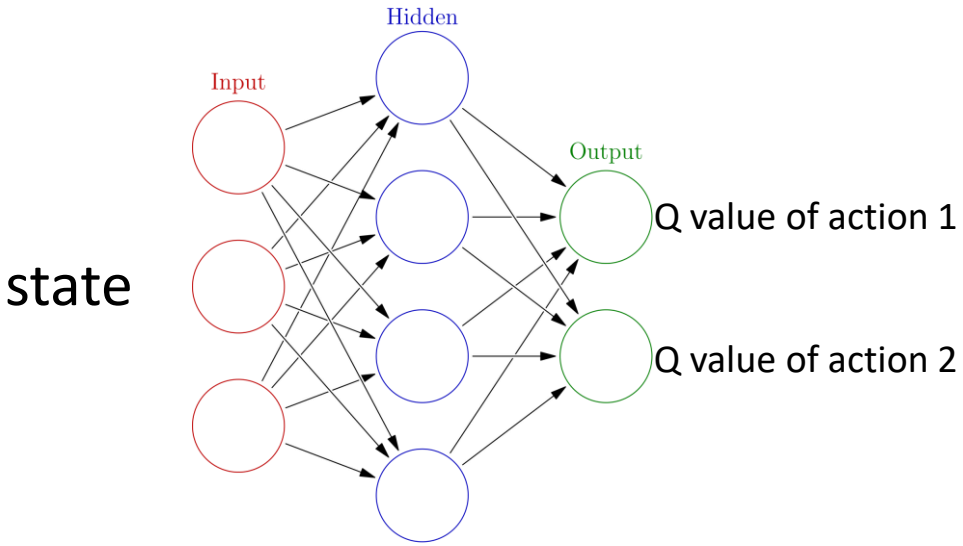
$$Q^{*}(s,a) = r(s,a) + \gamma \max_{a'} \mathbb{E}_{s' \sim p(\cdot|s,a)}[Q^{\pi}(s',a')]$$

Q-learning loss!

$$\min_{\theta} \sum_{t} \left(r(s_t,a_t) + \gamma \max_{a_{t+1}} \mathbb{E}_{s_{t+1} \sim p(\cdot|s_t,a_t)}[Q_{\theta}(s_{t+1},a_{t+1})] - Q_{\theta}(s_t,a_t) \right)^2$$

Basics of Q-learning

- We assume the action space is discrete.
- The Q-function can be formulated as a neural network like a classifier.
- The action to take is the one with the largest Q value.
- Training is performed by satisfying the Bellman's equation.



https://en.wikipedia.org/wiki/Neural_network_%28machine_learning%29

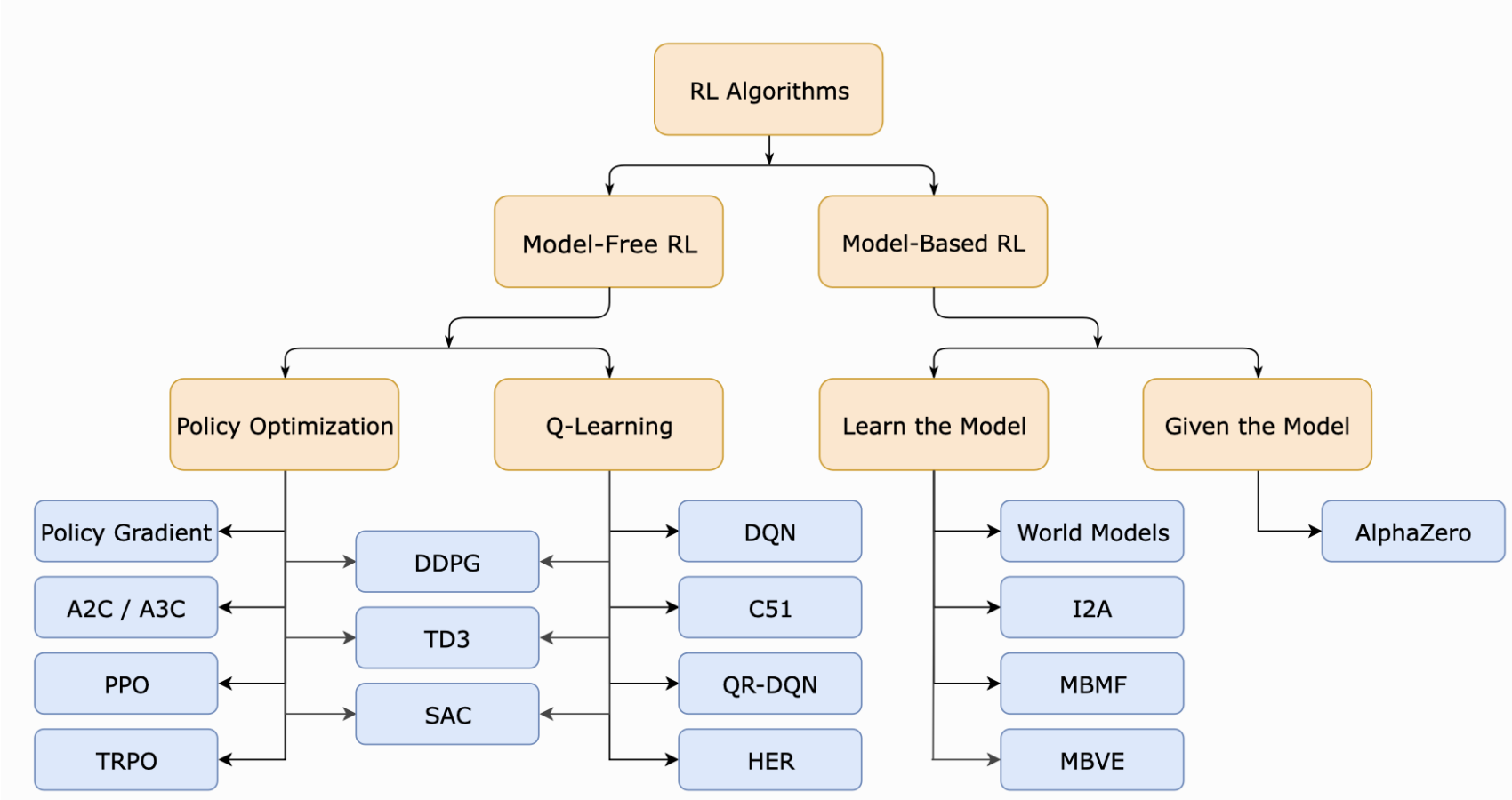
Key Concepts of RL

RL Learning Algorithms

Behavior Synthesis

Taxonomy
Policy Gradient
Actor-Critic Method
Proximity Policy Optimization

Learning algorithms taxonomy



https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html

Learning algorithms taxonomy

- **Online vs offline**
  - Online RL: The agent collects new data from interacting with the world.
  - Offline RL: The dataset is fixed, which might be chaotic.
- **On-policy vs off-policy**
  - On-policy: The agent uses a single policy (usually the newest one) to interact with the world. Old data is discarded.
  - Off-policy: The agent can use multiple policies to interact with the world. Old data is stored in a replay buffer and can be re-used.

Learning algorithms

- **Policy optimization**
  - Policy gradient
  - Actor-critic method (A2C)
  - Proximity policy optimization (PPO)
- **Q-learning (for self-study)**
  - Deep Q-learning (DQN)
  - Deep deterministic policy gradient (DDPG)
  - Soft actor-critic (SAC)

Policy gradient

- The policy network is parameterized by a neural network, and we aim to maximize the expected return

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] = \int_\tau p(\tau|\pi_\theta) R(\tau) d\tau$$

- Ideally, the learning can be performed by gradient ascent, e.g.

$$\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\pi_\theta)$$

- How to compute the gradient of the loss?

Policy gradient

- The loss gradient is given by

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau) \right]$$

- This can be computed empirically.
  1. Let the agent interact with the world based on the current policy.
  2. Collect a set of RL trajectories.
  3. Compute the returns and the policy gradients.
  4. Average.

Policy gradient

- How to derive the gradient?

Start with probability of RL trajectory

$$p(\tau|\pi) = p(s_0)\prod_t p(s_{t+1}|s_t, a_t)\pi(a_t|s_t)$$

Take log at both sides, and get

$$\log p(\tau|\pi_\theta) = \log p(s_0) + \sum_t \left(\log p(s_{t+1}|s_t, a_t) + \log \pi_\theta(a_t|s_t)\right)$$

Take gradient at both sides, and get

$$\nabla_\theta \log p(\tau|\pi_\theta) = \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

Policy gradient

- How to derive the gradient?

Start with the expected return

$$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \int_\tau p(\tau|\pi_\theta)R(\tau)d\tau$$

Take derivative at both sides,

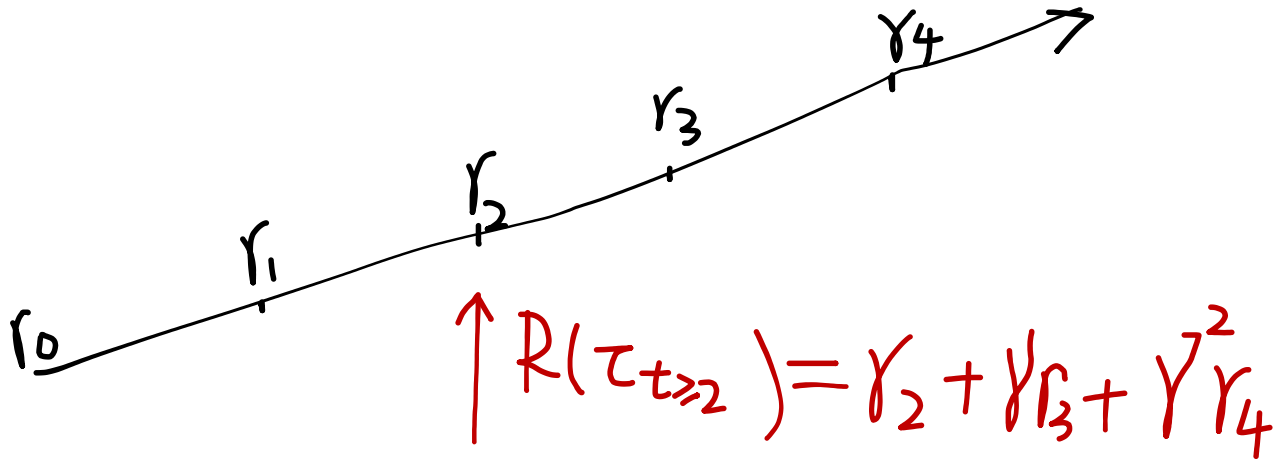
$$\nabla_\theta J(\pi_\theta) = \nabla_\theta \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \int_\tau \nabla_\theta p(\tau|\pi_\theta)R(\tau)d\tau$$

Apply log derivative trick:

$$\begin{aligned}
\nabla_\theta J(\pi_\theta) &= \int_\tau \nabla_\theta p(\tau|\pi_\theta)R(\tau)d\tau \\
&= \int_\tau p(\tau|\pi_\theta)\nabla_\theta \log p(\tau|\pi_\theta)R(\tau)d\tau \\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta \log p(\tau|\pi_\theta)R(\tau)\right] \\
&= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)R(\tau)\right]
\end{aligned}$$

Policy gradient

- The agent behaves in a causal way.
- In the same rollout, the rewards in the past does not influence the behavior in the future.
- To update the policy at time t, we only consider the return-to-go.

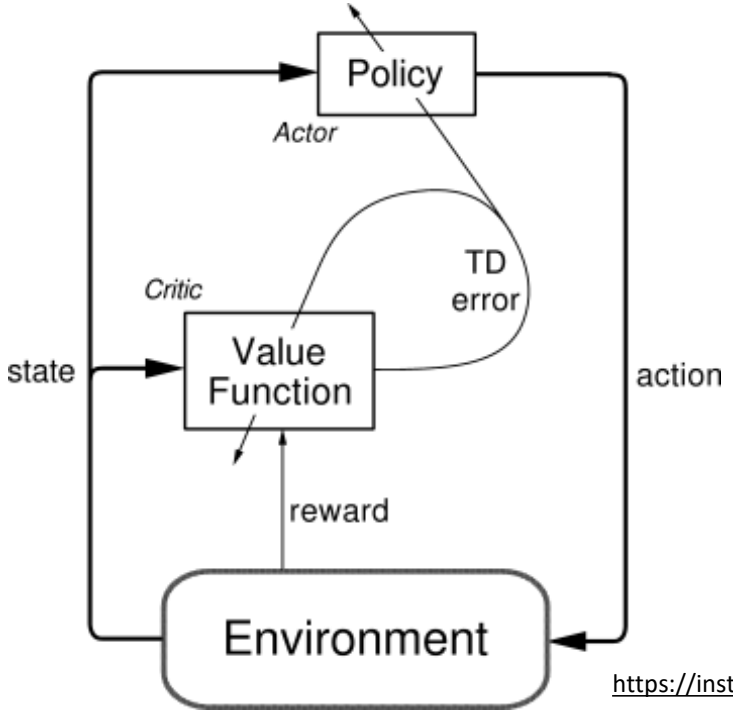


$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau)\right]$$

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau_{t'\geq t})\right]$$

Actor-critic method (A2C)

- This vanilla policy gradient is hard to use in practice because of high variance, making the training process unstable and easy to diverge.
- The A2C method is a solution for this issue. A very common in practice!



https://inst.eecs.berkeley.edu/~cs188/sp20/assets/files/SuttonBartoIPRLBook2ndEd.pdf

Actor-critic method (A2C)

- Why the policy gradient has high variance?

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau_{t' \geq t}) \right]$$

- Solution 1: Introduce a baseline to the gradient.
- Solution 2: replace the return-to-go with its expectation.

Solution 1: Policy gradient with baselines

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau_{t'\geq t})\right] \Rightarrow \nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \left(R(\tau_{t'\geq t}) - b(s_t)\right)\right]$$

- We wish reduce the gradient variance without introducing bias to its expectation.
- This can be satisfied if the baseline is a function of its current state.

Solution 1: Policy gradient with baselines

Expected Grad-Log-Prob (EGLP) lemma.

$$\int_x p_\theta(x)dx = 1 \quad \Rightarrow \quad \mathbb{E}_{x\sim p_\theta}[\nabla_\theta p_\theta(x)] = 0$$

Apply this lemma to policy gradient:

$$\mathbb{E}_{a_t\sim\pi_\theta}[\nabla_\theta \pi_\theta(a_t|s_t)] = 0$$

Multiply an arbitrary baseline that is only dependent on the state:

$$\mathbb{E}_{a_t\sim\pi_\theta}[\nabla_\theta \pi_\theta(a_t|s_t)b(s_t)] = 0$$

Solution 1: Policy gradient with baselines

- The most common choice is the on-policy value function.

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \left(R(\tau_{t' \geq t}) - b(s_t) \right) \right] \qquad b(s_t) = V^\pi(s_t)$$

- We use another neural network to parameterize the value function, and learn it by minimize

$$\min_\phi \mathbb{E}_{\tau \sim \pi_\theta} \left[\left(R(\tau_{t' \geq t}) - V_\phi(s_t) \right)^2 \right]$$

Solution 2: Replace return-to-go with its expectation

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) R(\tau_{t'\geq t})\right]$$

$$= \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) Q^{\pi_\theta}(s_t, a_t)\right]$$

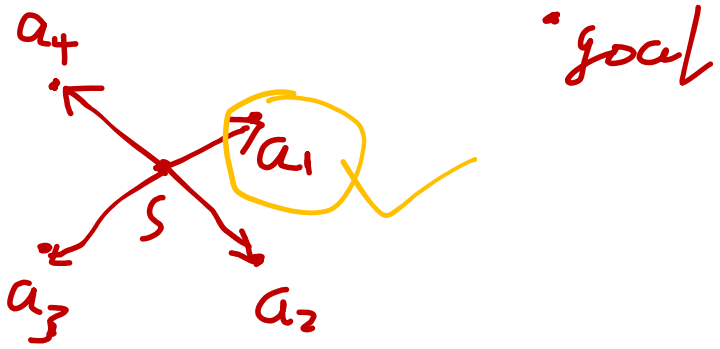
Actor-critic method (A2C)

- When combining both solutions, we can derive

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) A^{\pi_\theta}(s_t, a_t) \right]$$

$$A^{\pi_\theta}(s_t, a_t) = Q^{\pi_\theta}(s_t, a_t) - V^{\pi_\theta}(s_t)$$

- The difference between Q-function and the state value function is called advantage function.

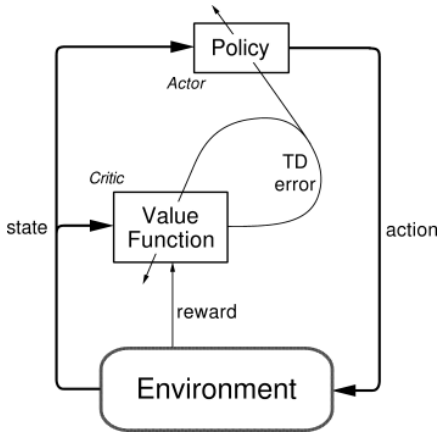


Actor-critic method (A2C)

- We have three networks to learn: the policy network, the Q-function, and the state value function.
- Based on the Bellman equation, the Q-function can be approximated by the time difference of the state value function!

$$
\begin{aligned}
Q^{\pi}(s,a) &= r(s,a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,a), a' \sim \pi}[Q^{\pi}(s',a')] \\
&= r(s,a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,a)}[V^{\pi}(s')]
\end{aligned}
$$

Actor-critic method (A2C)



- The advantage function is approximated by

$$A^{\pi_\theta}(s_t, a_t) \approx r_t + \gamma V^{\pi_\theta}(s_{t+1}) - V^{\pi_\theta}(s_t)$$

- The policy network is updated via gradient ascent, with the gradient

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) A^{\pi_\theta}(s_t, a_t)\right]$$

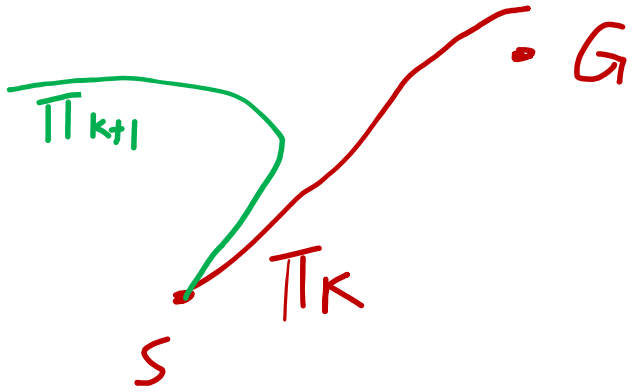
- The value network is updated by the Bellman equation constraint, e.g.

$$\min_\phi \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t \left(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)\right)^2\right]$$

can be from old V_ϕ estimate

Proximity Policy Optimization (PPO)

- Policy gradient ascent is still not stable.
- Inappropriate parameter update will change the policy a lot.
- This would collapse training or make data collection very inefficient.
- Therefore, we would like to update the policy in a more effective way.



Proximity Policy Optimization (PPO)

- Instead of maximizing the expected return directly, a surrogate advantage objective is proposed.

$$L(\theta, \theta_k) = \mathbb{E}_{s,t,a\sim\pi_k}\left[\min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}A_t^{\pi_{\theta_k}}, \text{clip}\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1-\epsilon, 1+\epsilon\right)A_t^{\pi_{\theta_k}}\right)\right]$$

- By maximizing the surrogate objective, the policy is updated stably.
- Please see TRPO and PPO papers for more details.

Proximity Policy Optimization (PPO)

- surrogate objective is maximized, instead of the expected return directly.

Policy Gradient $$J(\pi_\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)] = \int_\tau p(\tau|\pi_\theta)R(\tau)d\tau$$

TRPO $$L(\theta,\theta_k) = \mathbb{E}_{s,t,a\sim\pi_k}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}A_t^{\pi_{\theta_k}}\right] \quad \text{s.t. } \bar{D}_{KL}(\theta||\theta_k) \le \delta$$

PPO $$L(\theta,\theta_k) = \mathbb{E}_{s,t,a\sim\pi_k}\left[\min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}A_t^{\pi_{\theta_k}}, \text{clip}\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1-\epsilon, 1+\epsilon\right)A_t^{\pi_{\theta_k}}\right)\right]$$

Proximity Policy Optimization (PPO)

Algorithm 1 PPO-Clip

1: Input: initial policy parameters θ_0, initial value function parameters ϕ_0
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
4: Compute rewards-to-go $\hat{R}_t$.
5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.
6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.
7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.
8: **end for**

https://spinningup.openai.com/en/latest/algorithms/ppo.html

Key Concepts of RL

RL Learning Algorithms

Behavior Synthesis

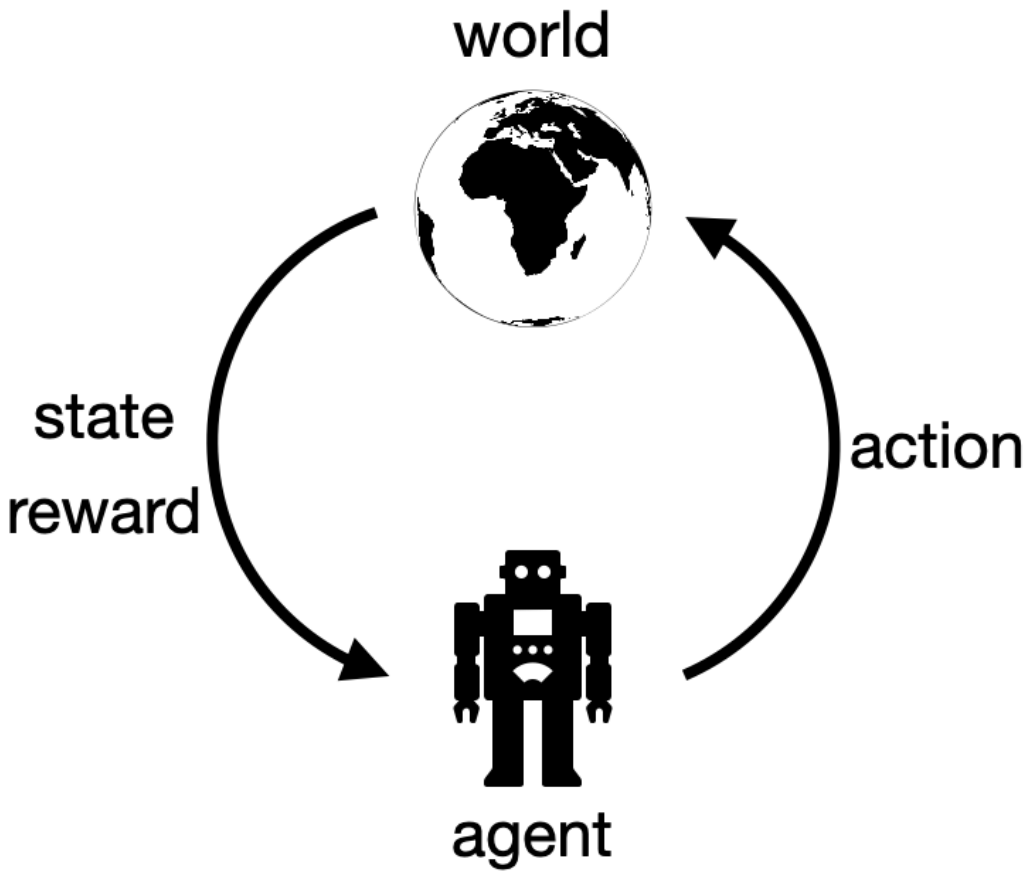
Path-following

Inhabiting

Human Ego-Sensing

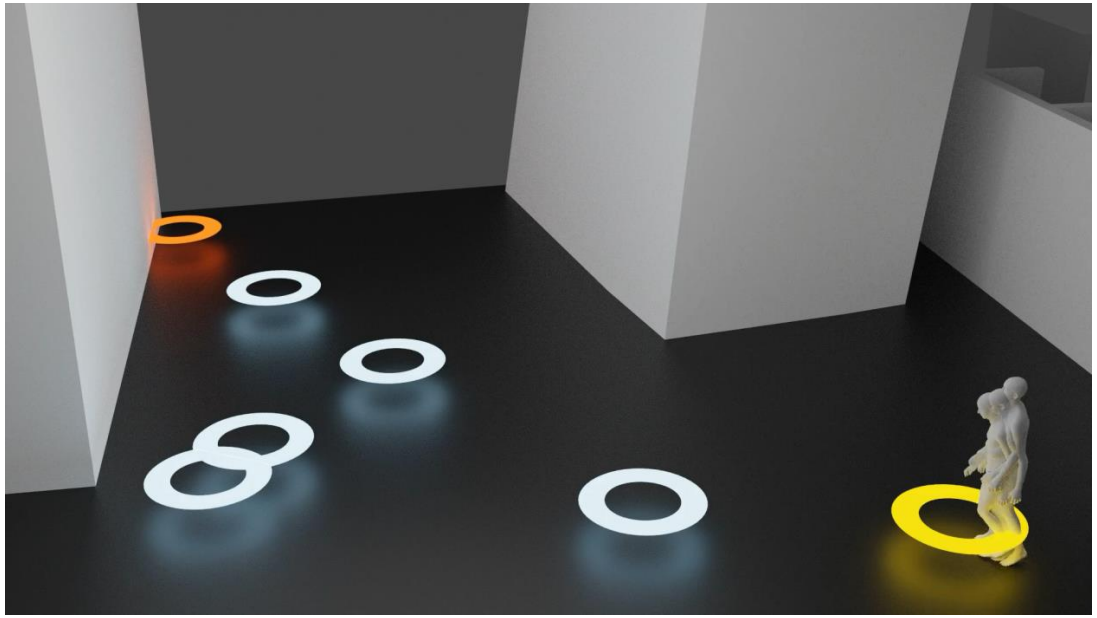
How to model behavior?

1. Define the agent.
2. Define the world.
3. Define the action space.
   - E.g. Continuous or discrete?
4. Define the reward.
   - What is the goal and how to evaluate the agent performance?
5. Define the state.
   - What the agent should perceive in the world?
   - Should be relevant to the goal, as the agent will make decision based on the perception.
6. Learn the policy.



How to model path-following behavior?

1. Define the agent.
2. Define the world.
3. Define the action space.
   - E.g. Continuous or discrete?
4. Define the reward.
   - What is the goal and how to evaluate the agent performance?
5. Define the state.
   - What the agent should perceive in the world?
   - Should be relevant to the goal, as the agent will make decision based on the perception.
6. Learn the policy.



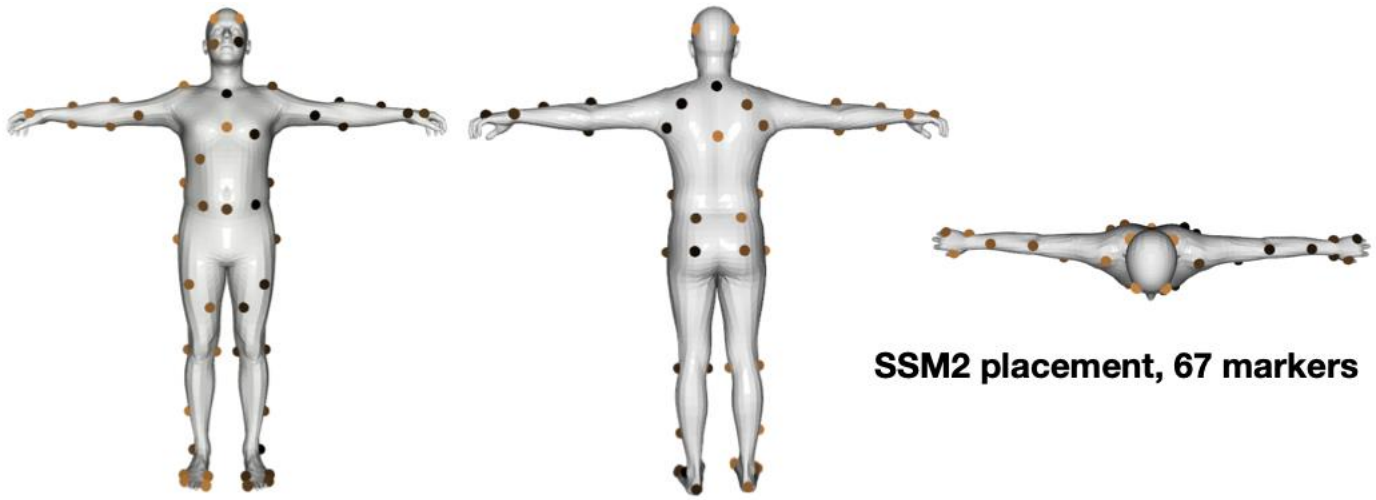
Define the agent

- We use the parametric human body model SMPL-X [Pavlakos et al., SMPL-X, 2009].
- It takes as input the gender, the body shape parameters and the pose parameters as input, and produces a body mesh with 10475 vertices.
- SMPL-X is widely used in various applications such as 3D pose estimation and avatar appearance capture.

[Pavlakos et al., SMPL-X, 2009]
https://ps.is.mpg.de/uploads_file/attachment/attachment/497/SMPL-X.pdf

Define the agent

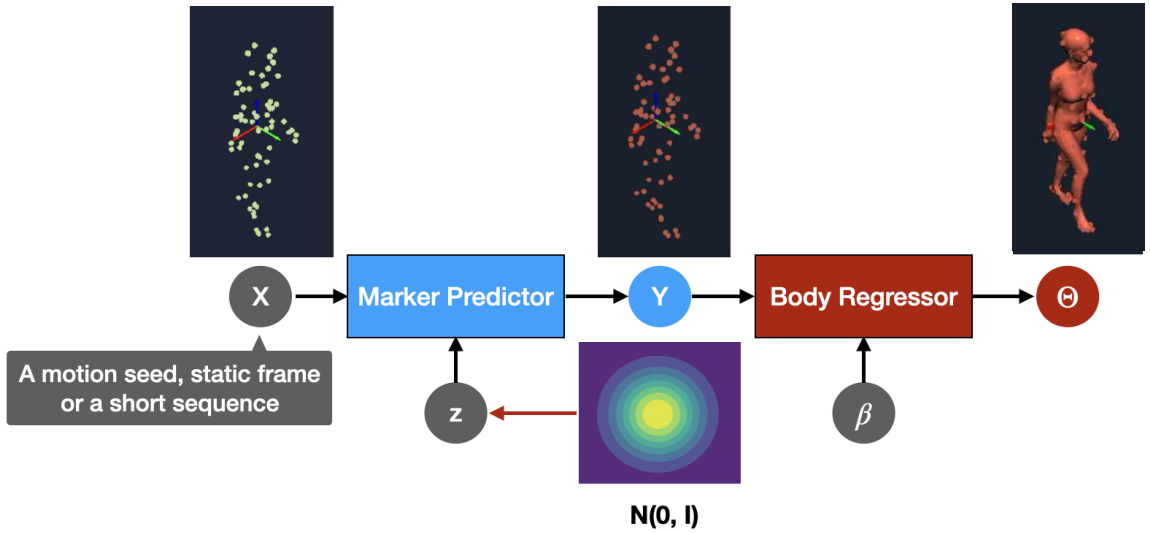
- We need to represent how the agent moves in the 3D world.
- Inspired by marker-based mocap, we use virtual markers on the body mesh template to obtain a compact body representation.
- With this virtual markers, body degrees of freedom and shape can be represented.



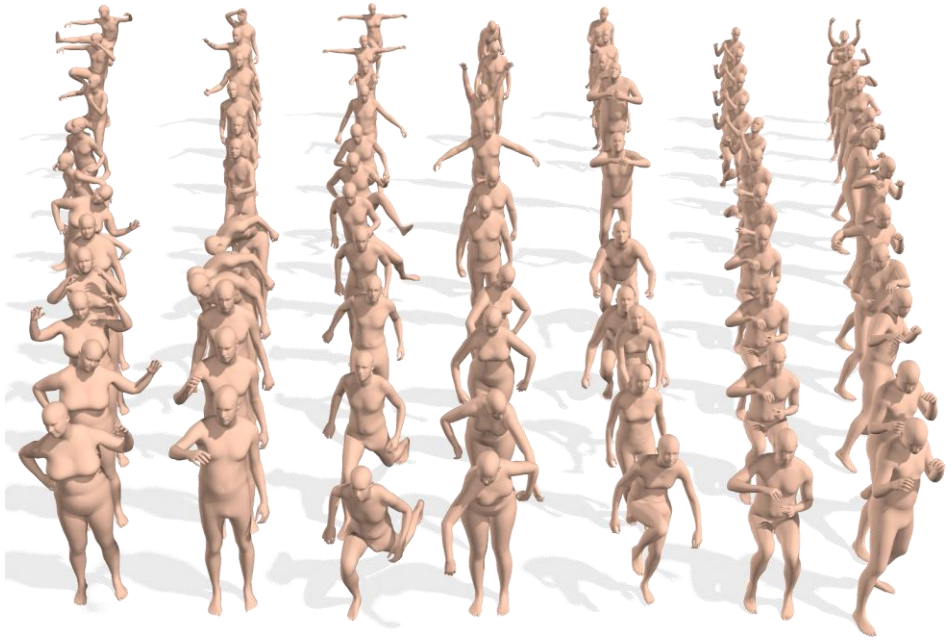
[Zhang et al., MOJO, 2021]

Define the world

- We need to define the transition probability to model how bodies move.
- We learn a generative model of human motions from AMASS.



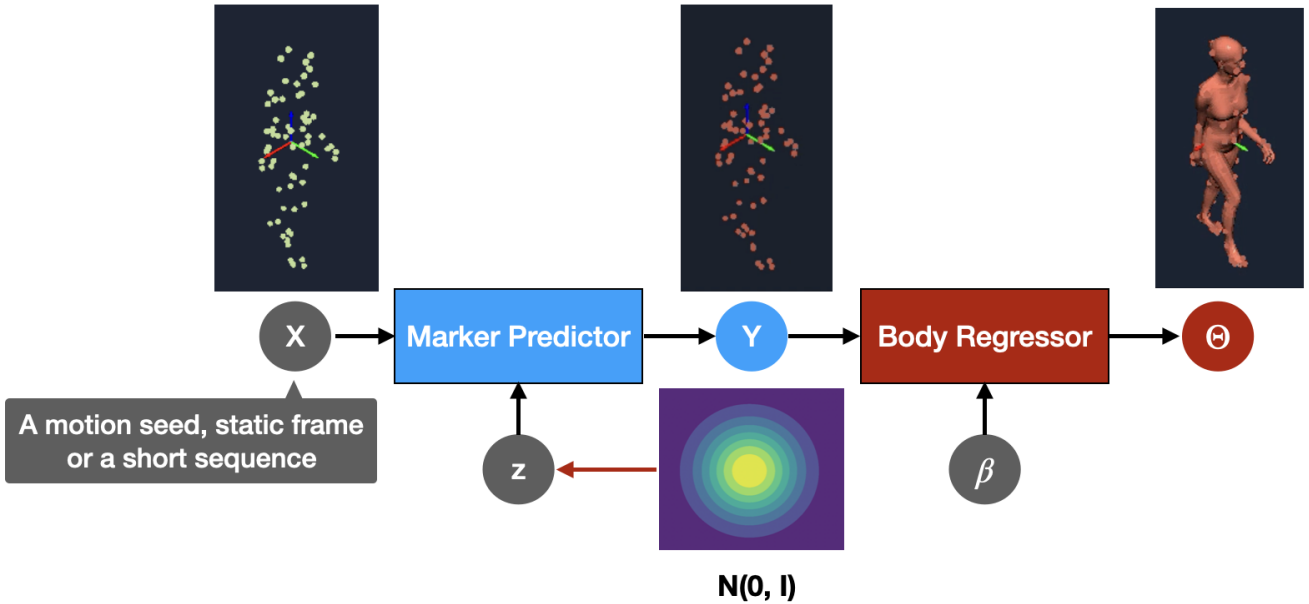
[Zhang and Tang, GAMMA, 2022]



[Mahmood et al., AMASS, 2019]
https://amass.is.tue.mpg.de

Define the action space

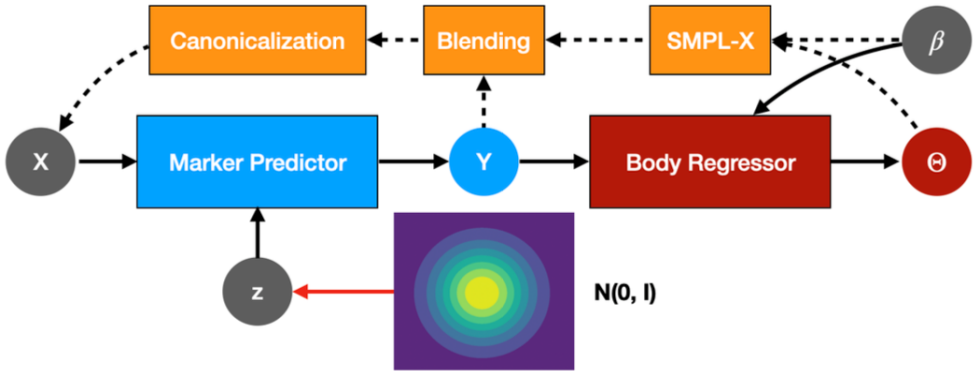
- Usually, if the agent and the world are specified, the action space is determined.
- In this case, the action is the latent variable of the pre-trained motion model, and the action space is the standard normal distribution.

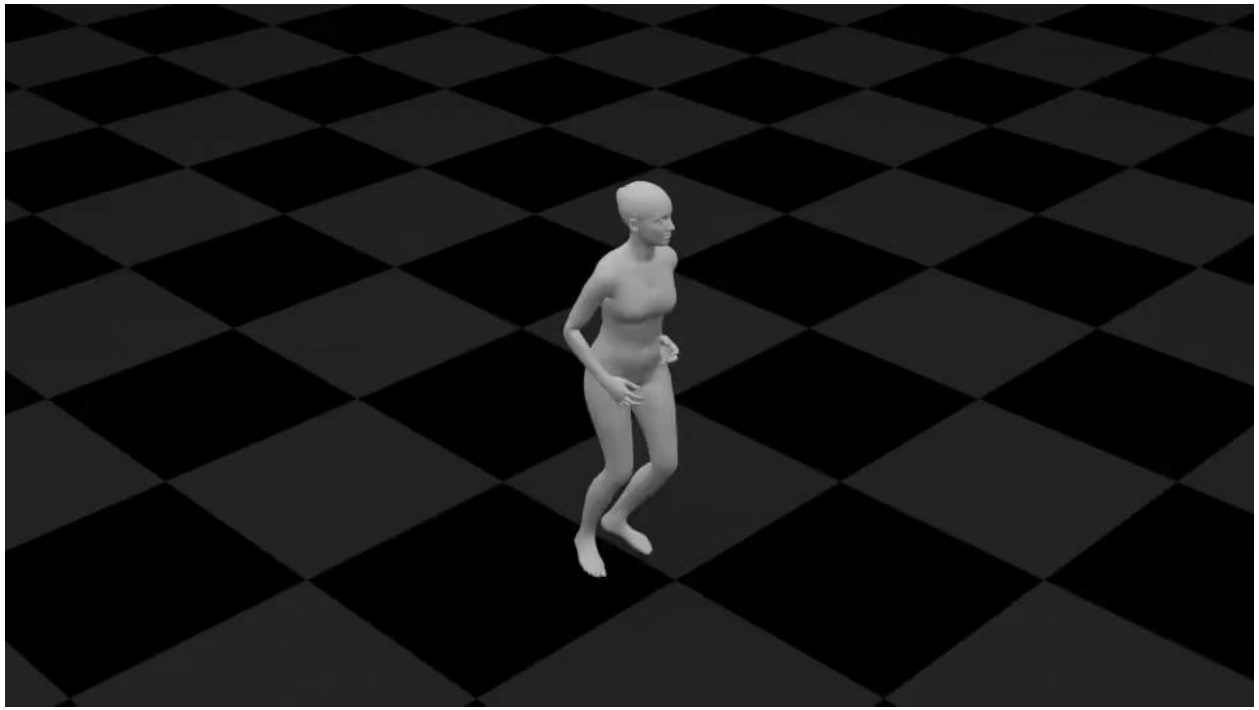


[Zhang and Tang, GAMMA, 2022]

Define the action space

- After specifying the action space, the agent can move without goals.
- This is realized by employing the motion model recursively.

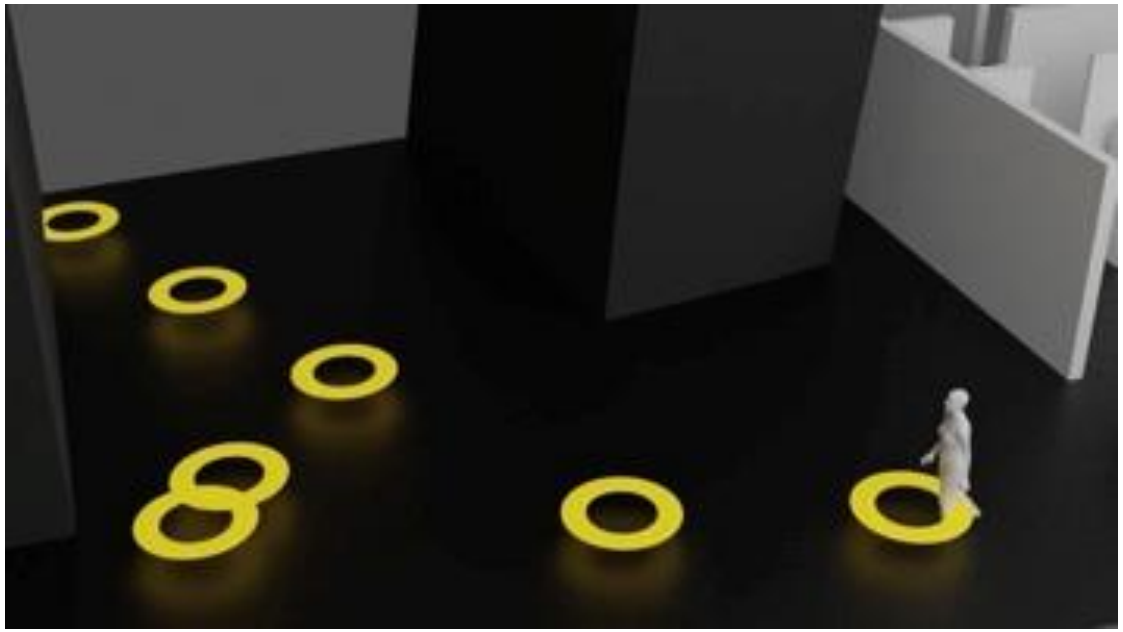




[Zhang and Tang, GAMMA, 2022]

Define the goal and the rewards

- To model path-following behavior, we desire the agent can reach individual waypoints, while keeping the motion realistic.



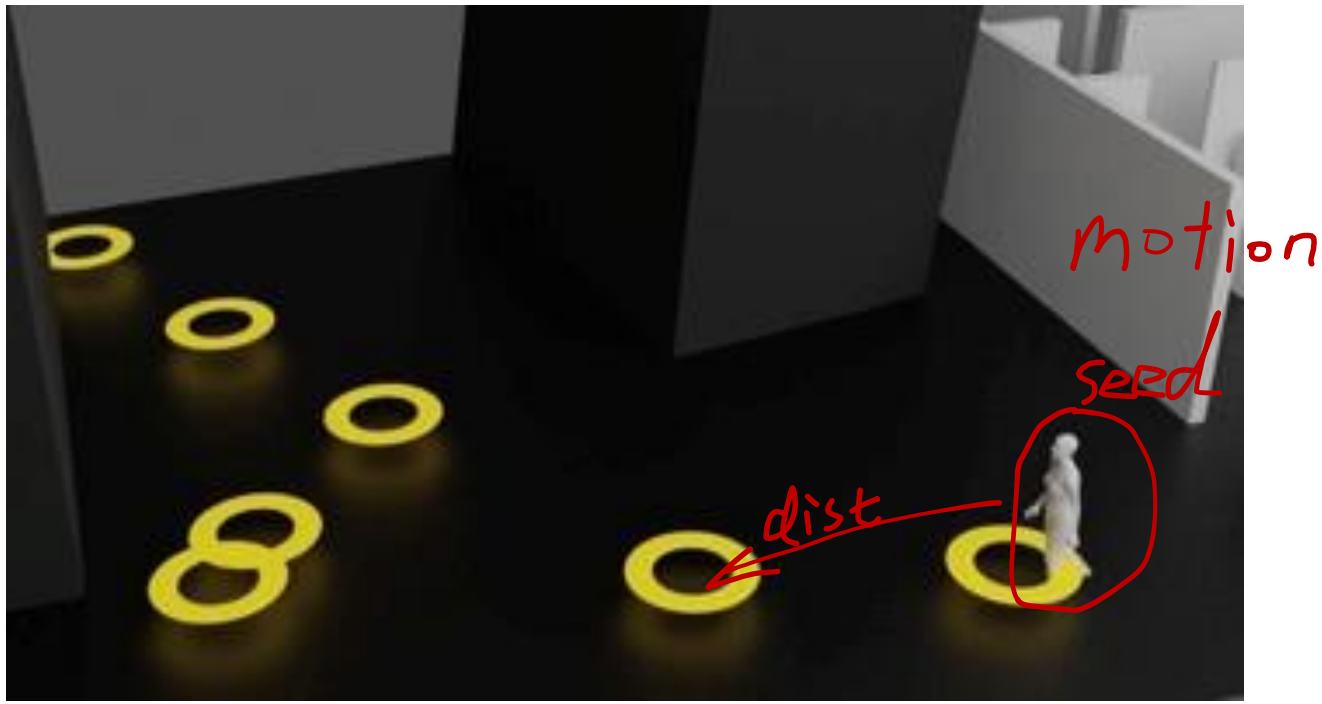
Rewards $r = r_{path} + \beta_1 r_{ori} + \beta_2 r_{contact} + \beta_3 r_{vposer} + \beta_4 r_{goal}$

Encourage moving straight to the goal.	Encourage plausible body-ground contact.	Encourage natural body poses.
Encourage facing the goal.	Bonus to all primitives in the motion if the goal is reached.	

[Zhang and Tang, GAMMA, 2022]

Define the state

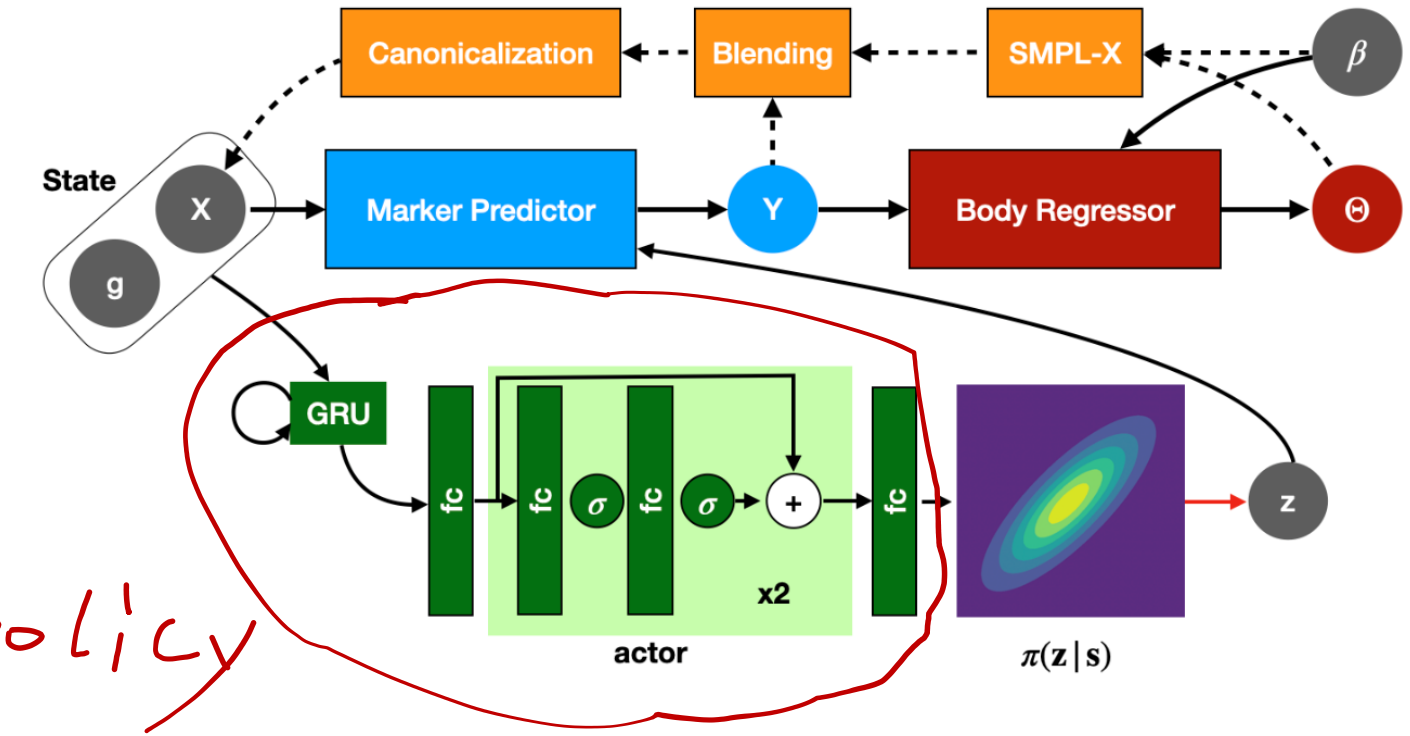
- The state indicates the agent's perception and is relevant to the goal.
- The state contains the current motion seed and the distance to the next waypoint.



[Zhang and Tang, GAMMA, 2022]

Learn the policy

- The policy network takes the state as input and produces the action.
- Instead of sampling from N(0,I), we sample from the policy output.



[Zhang and Tang, GAMMA, 2022]

Learn the policy

- We perform on-policy learning.
- The policy is learned by minimizing the following loss.
- Data collection and policy update are alternating.

$$\mathcal{L}_{policy} = \mathcal{L}_{PPO} + \mathbb{E}\left[(R_t - V(\boldsymbol{s}_t))^2\right] + \alpha\Psi\left(\text{KL-div}(\pi(\boldsymbol{z}|\boldsymbol{s})||\mathcal{N}(0, \boldsymbol{I}))\right)$$

Policy updating with PPO

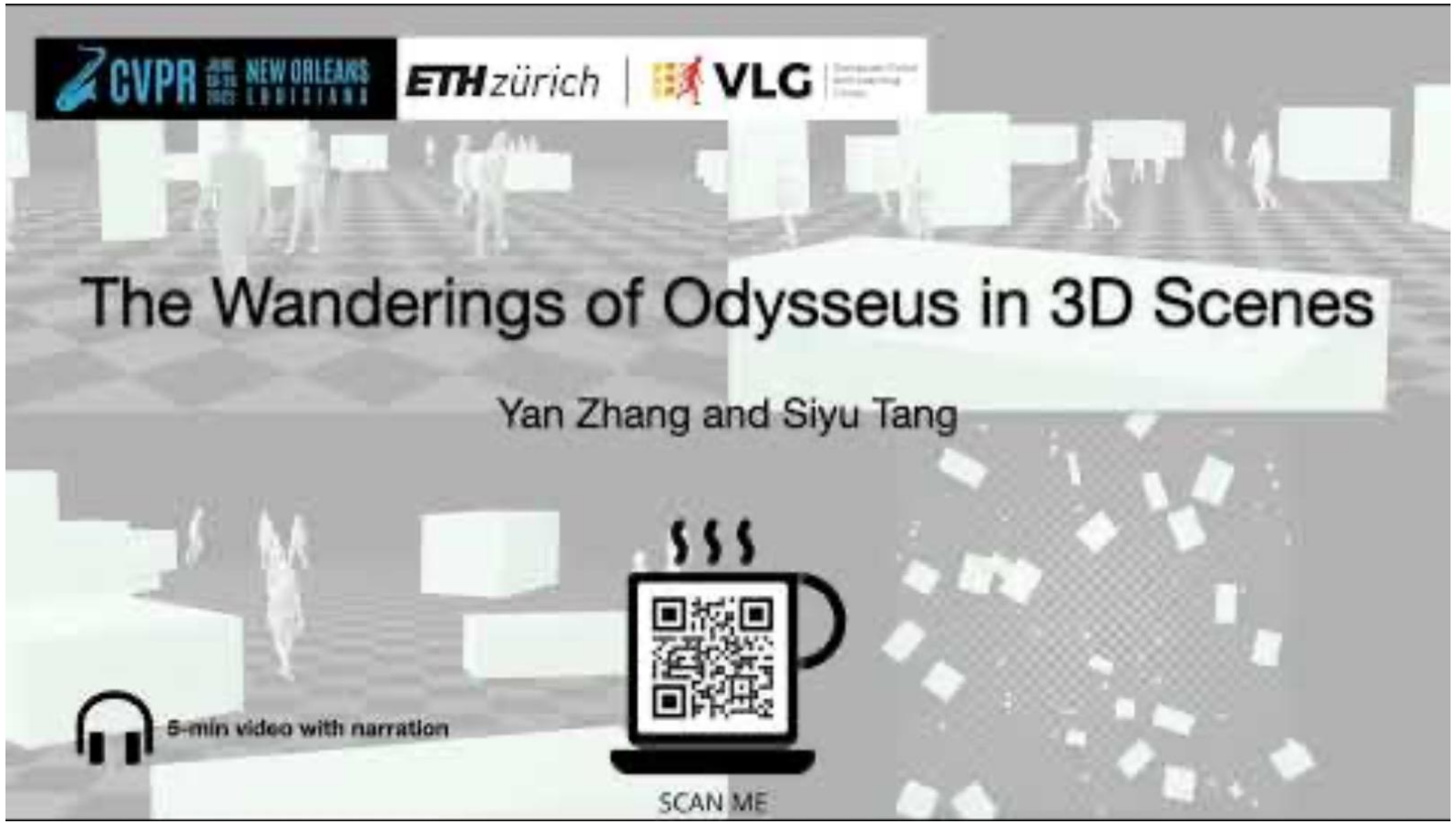
Value function estimation

Motion prior to encourage motion realism

[Zhang and Tang, GAMMA, 2022]

Learn the policy

- The learning process is as follows.



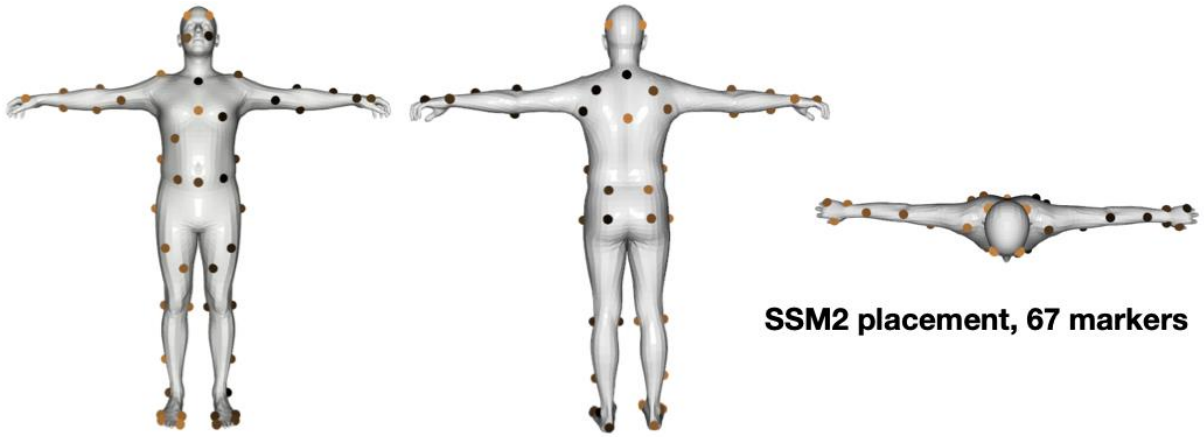
[Zhang and Tang, GAMMA, 2022]

Inhabiting

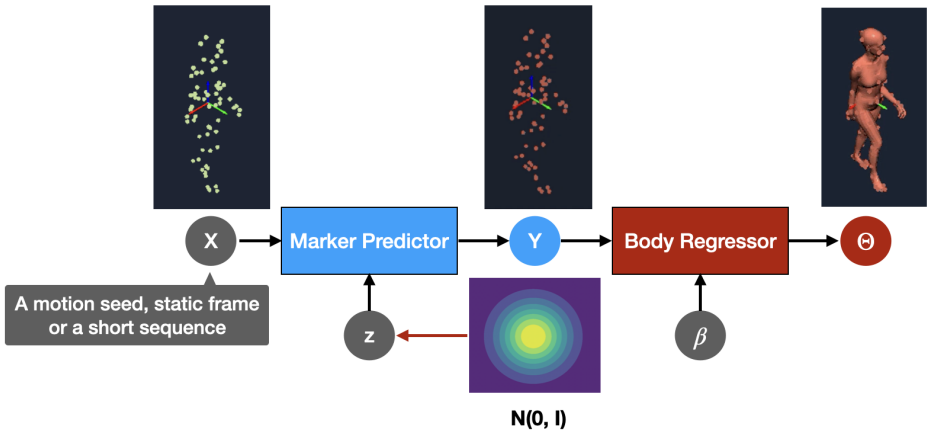
- How to model inhabiting behavior?
  1. Define the agent.
  2. Define the world.
  3. Define the action space.
  4. Define the reward.
  5. Define the state.
  6. Learn the policy.

[Zhao et al., DIMOS, 2023]

Define the agent, the world, and the action space



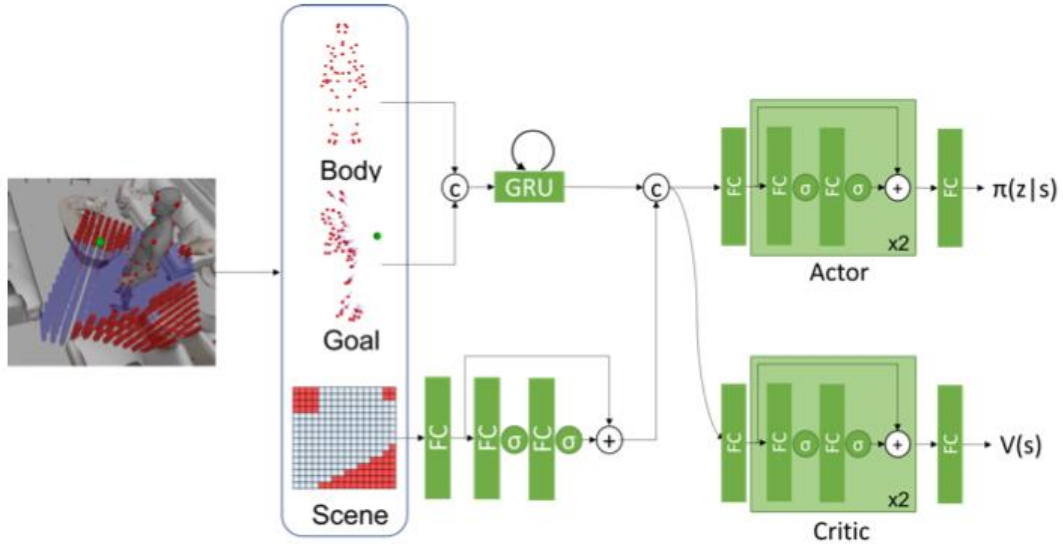
agent



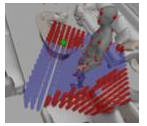
The transition probability, additionally trained human-scene interaction mocap sequences.

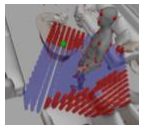
The states and the policies

- The states and policies are defined respectively for different human-scene interactions.



Locomotion in the 3D scene

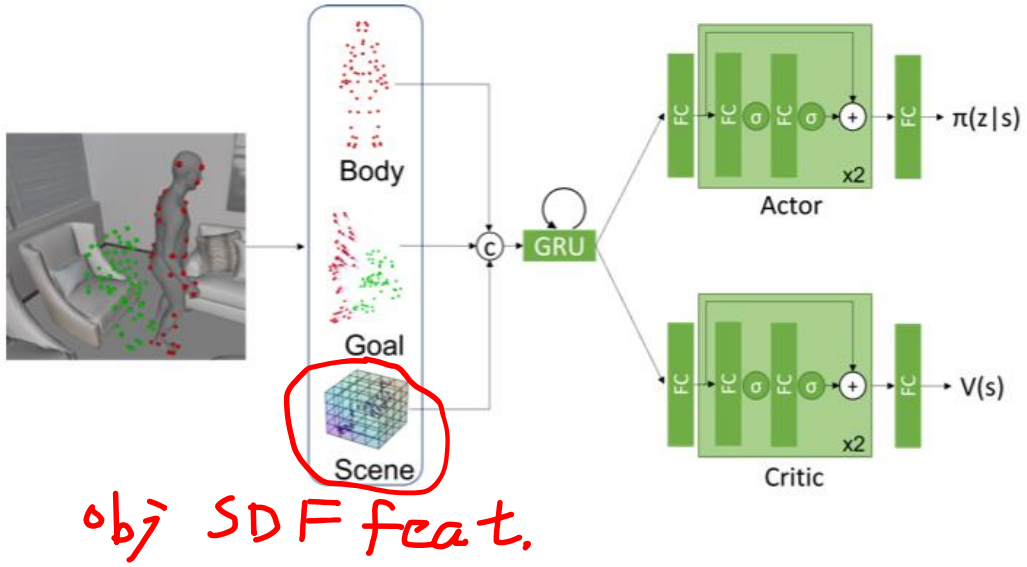
without 

with 

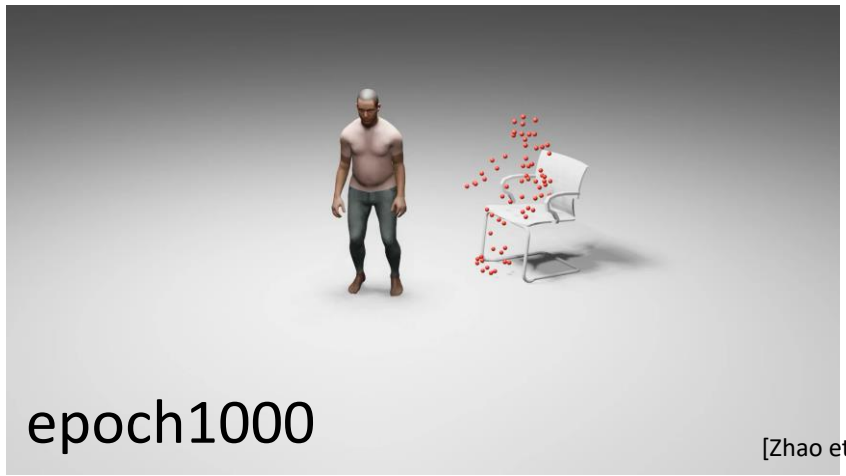
[Zhao et al., DIMOS, 2023]

The states and the policies

- The states and policies are defined respectively for different human-scene interactions.



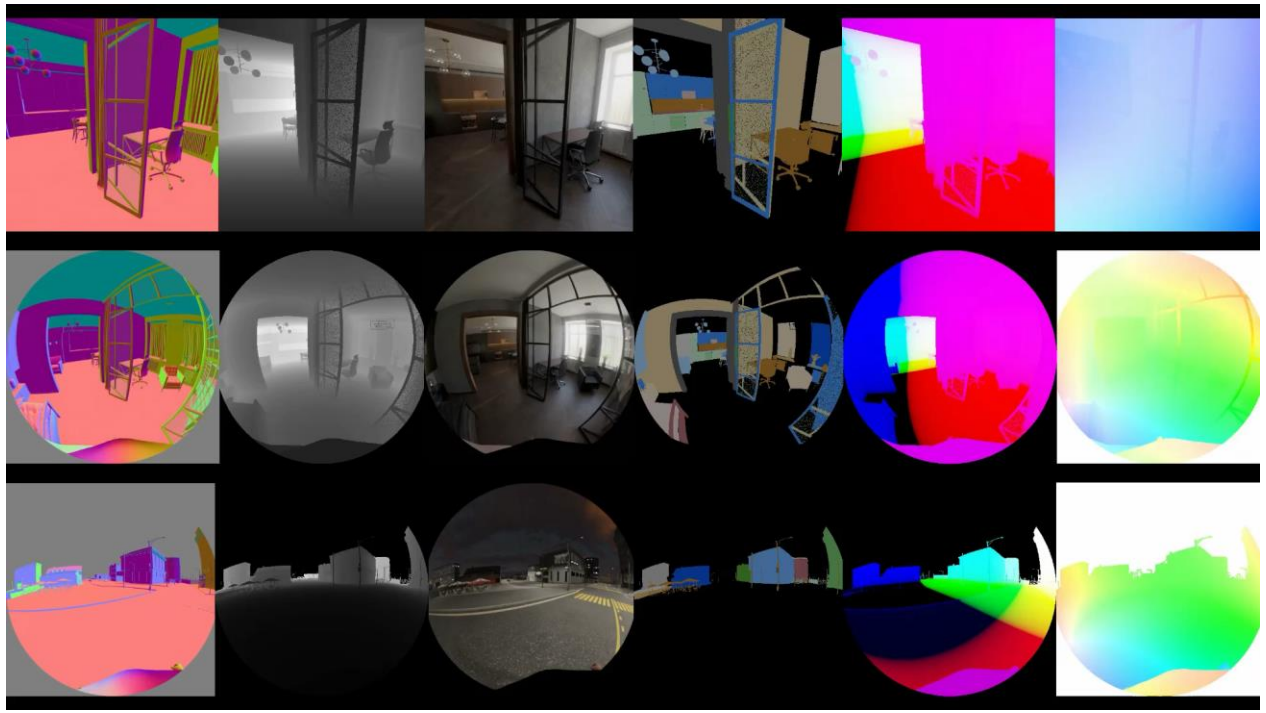
Fine-grained human-scene interactions



[Zhao et al., DIMOS, 2023]

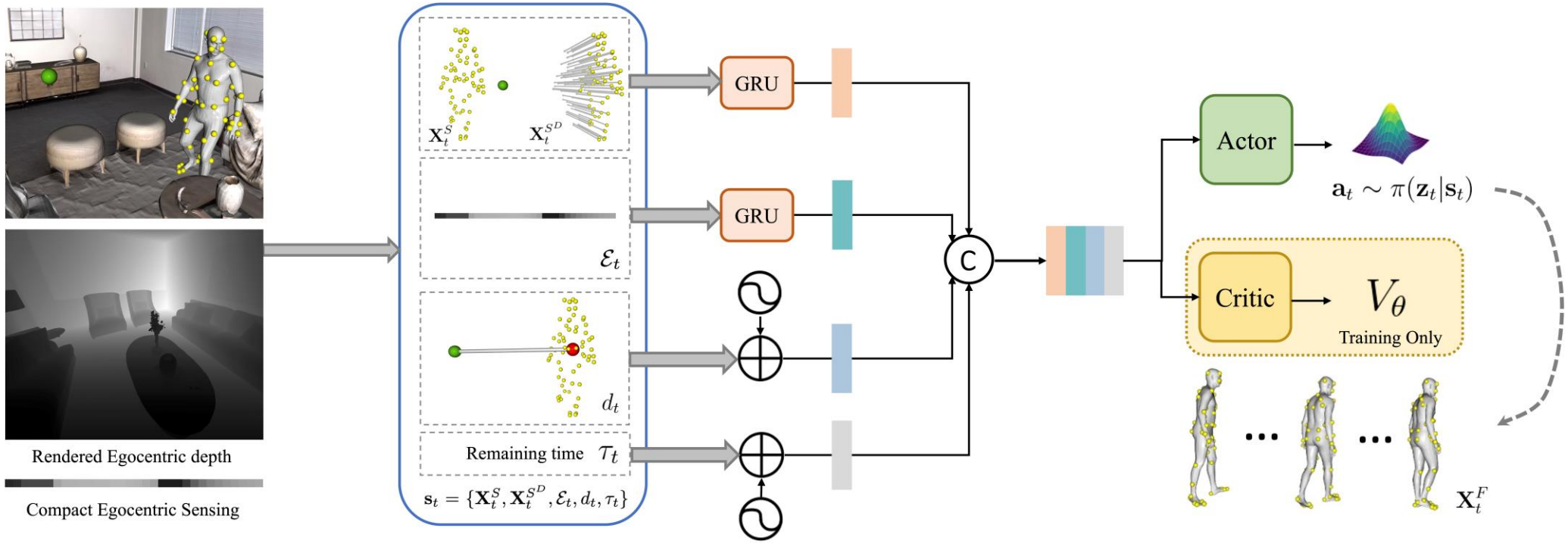
How to simulate human egocentric sensing?

1. Define the agent.
2. Define the world.
3. Define the action space.
4. Define the reward.
5. Define the state.
6. Learn the policy.



[Li et al., EgoGen, 2024]

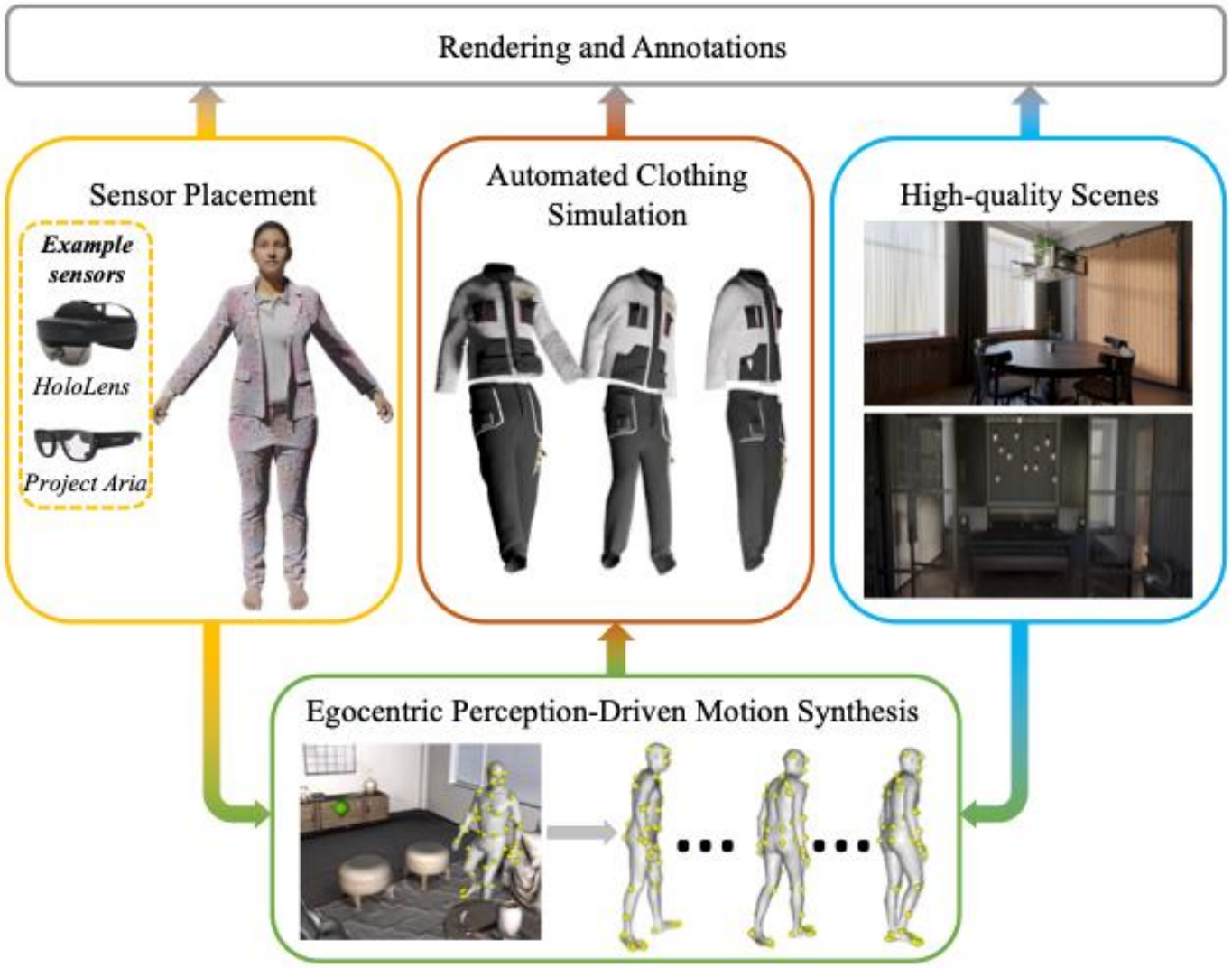
The states, policies, and rewards



A reward is added to encourage attention to (look at) the goal.

[Li et al., EgoGen, 2024]

Human Ego-Sensing

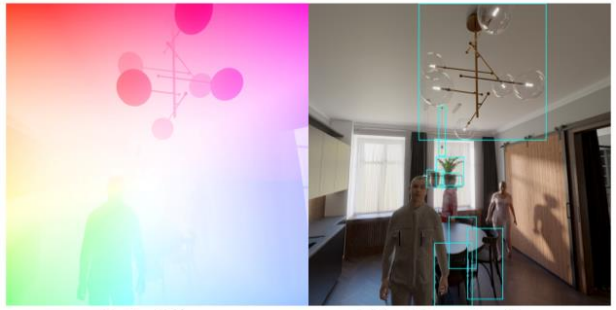


[Li et al., EgoGen, 2024]

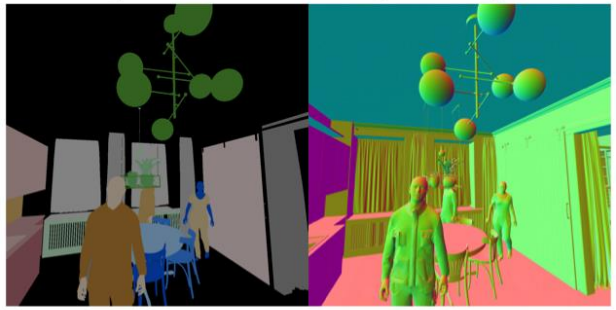
Human Ego-Sensing


RGB

Depth

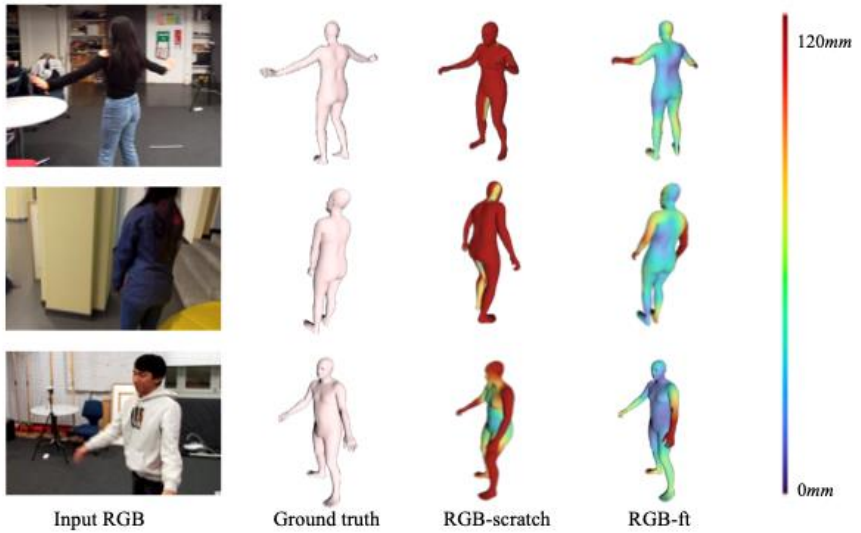

Optical Flow

Object Bounding Boxes


Segmentation Mask

Surface Normal

Synthetic data



(b) Human Mesh Recovery from RGB Images

Table 6. Evaluation of HMR on EgoBody test set. "*-scratch" denotes the model trained from scratch with the Egobody training set, and "*-ft" denotes the model pre-trained with *EgoGen* synthetic data. The units for all metrics are in *mm*. (Sec. 5.5)

	G-MPJPE ↓	MPJPE ↓	PA-MPJPE ↓	V2V ↓
Depth-scratch	117.7	82.2	54.1	100.6
Depth-ft	**90.7**	**65.2**	**47.3**	**81.0**
RGB-scratch	-	90.7	59.9	102.1
RGB-ft	-	**85.3**	**56.2**	**97.2**

Boost human mesh recovery

[Li et al., EgoGen, 2024]

Summary

Key concepts of RL

- Markov Decision Process (MDP), RL Trajectory
- state, action, reward, policy, return,
- value functions

RL learning algorithms

- Policy gradients
- Actor-critic method
- Proximity policy optimization

Behavior synthesis

- Walking Path following
- Inhabiting
- Human-centric sensing

References and Reading List

- OpenAI Spinning Up in Deep RL
  - https://spinningup.openai.com/en/latest/
- Course CS 285 at UC Berkeley
  - https://rail.eecs.berkeley.edu/deeprlcourse/
- Reinforcement Learning: An Introduction
  - http://www.incompleteideas.net/book/the-book.html

Bellman equations in optimum

$$Q^{\pi}(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim p(\cdot|s,a), a' \sim \pi}[Q^{\pi}(s',a')]$$

$$V^{*}(s) = \max_{a} \mathbb{E}_{s' \sim p(\cdot|s,a)}[r(s,a) + \gamma V^{*}(s')]$$

$$Q^{*}(s,a) = r(s,a) + \gamma \max_{a'} \mathbb{E}_{s' \sim p(\cdot|s,a)}[Q^{\pi}(s',a')]$$